CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: Austin Hunwardsen

Peter Knipper

Project Teams Group #: F_03

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

IF Stage (Instruction Fetch)

Category	Signals / Values
Datapath Values	PC, PC+4, Instruction, NextInstAddr, IMemAddr
Control Signals	UseNextAdr, Stall, iInstLd, iRST
Outputs to IF/ID	PC, PC+4, Instruction

ID Stage (Instruction Decode)

Category	Signals / Values
Inputs from IF/ID	PC, PC+4, Instruction
Generated Datapath Values	RS1Data, RS2Data, Immediate, RS1Addr, RS2Addr, RDAddr
Control Signals (Generated)	RegWrite, MemToReg, MemWrite, MemRead, Branch, ALUSrc, ALUOp
Outputs to ID/EX	All control signals, PC, PC+4, RS1Data, RS2Data, Immediate, Register Addresses

EX Stage (Execute)

Category	Signals / Values
Inputs from ID/EX	PC, PC+4, RS1Data, RS2Data, Immediate, RS1/RS2/RD addresses
Control Signals In	RegWrite, MemToReg, MemWrite, MemRead, Branch, ALUSrc, ALUOp
Generated Values	ALUResult, Zero, BranchAddr, ALUIn1, ALUIn2, ALUCtrl
Generated Control	BranchTaken, PCSrc, IsAUIPC, IsJAL, IsJALR
Outputs to EX/MEM	Control signals, PC+4, BranchAddr, ALUResult, RS2Data, RDAddr

MEM Stage (Memory Access)

Category	Signals / Values
Inputs from EX/MEM	PC+4, ALUResult, RS2Data, RDAddr
Control Signals In	RegWrite, MemToReg, MemWrite, MemRead
Generated Datapath Values	DMemOut, MemData (processed load value)
Memory Interface	DMemAddr, DMemData, DMemWr
Outputs to MEM/WB	Control signals, PC+4, ALUResult, MemData, RDAddr

WB Stage (Write Back)

Category	Signals / Values
Inputs from MEM/WB	PC+4, ALUResult, MemData, RDAddr
Control Signals In	RegWrite, MemToReg
Write Data Logic	If JAL/JALR → PC+4; else if MemToReg → MemData; else → ALUResult
Register File Write	RegWrite, RDAddr, WriteData

Pipeline Registers Summary

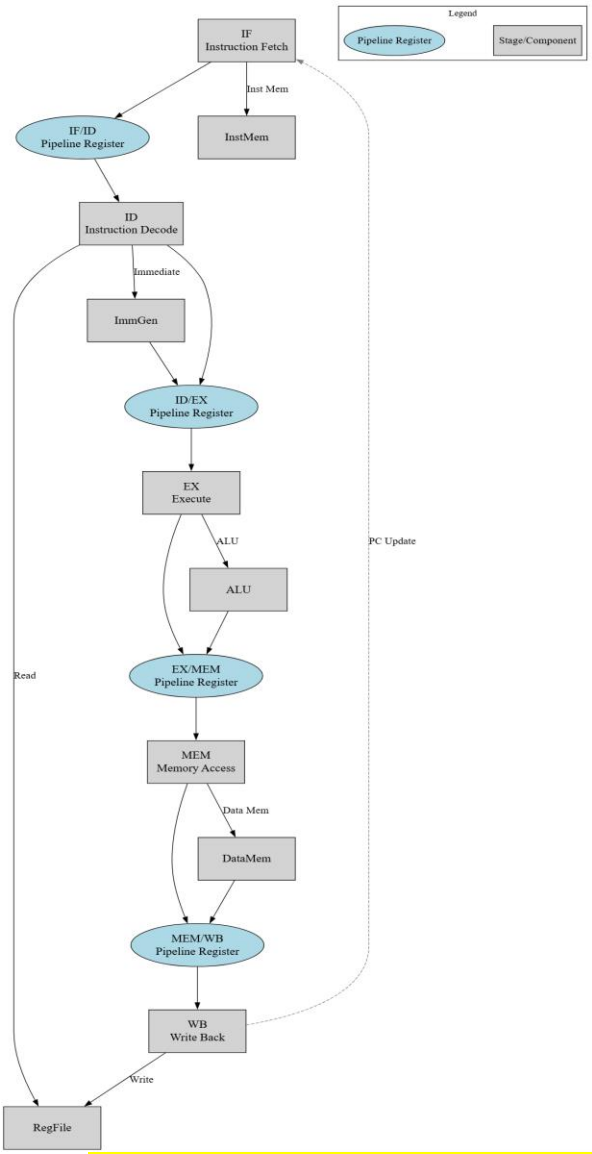
Pipeline Register	Inputs	Outputs	Controls
IF/ID	PC, PC+4, Instruction	PC, PC+4, Instruction	Write Enable, Flush

ID/EX	Control signals, PC, PC+4, RS1/RS2, Immediate, Addresses, Instruction	Same	Write Enable, Flush
EX/MEM	Control signals, PC+4, BranchAddr, PCSrc, ALUResult, RS2Data, RDAddr	Same	Write Enable, Flush
MEM/WB	Control signals, PC+4, ALUResult, MemData, RDAddr	Same	Write Enable, Flush

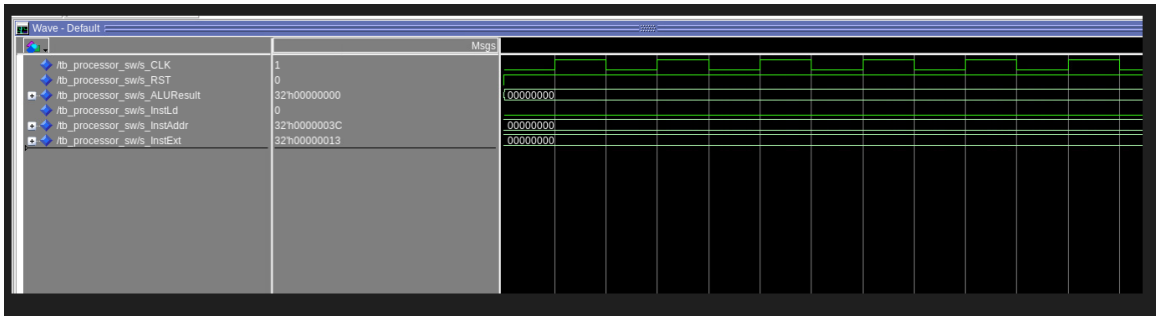
Critical Timing Paths

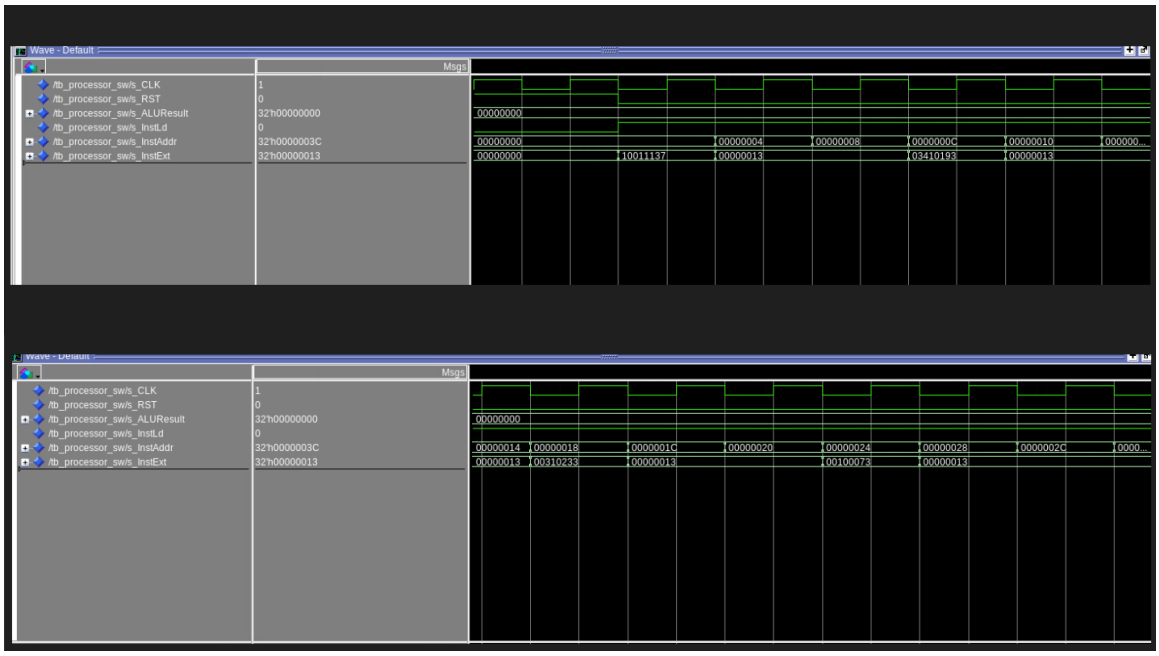
Stage	Critical Operations
IF	Instruction memory access
ID	Regfile read + control decode + immediate gen
EX	ALU + branch logic (**longest path**)
MEM	Data memory access
WB	Writeback mux + regfile write

[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.





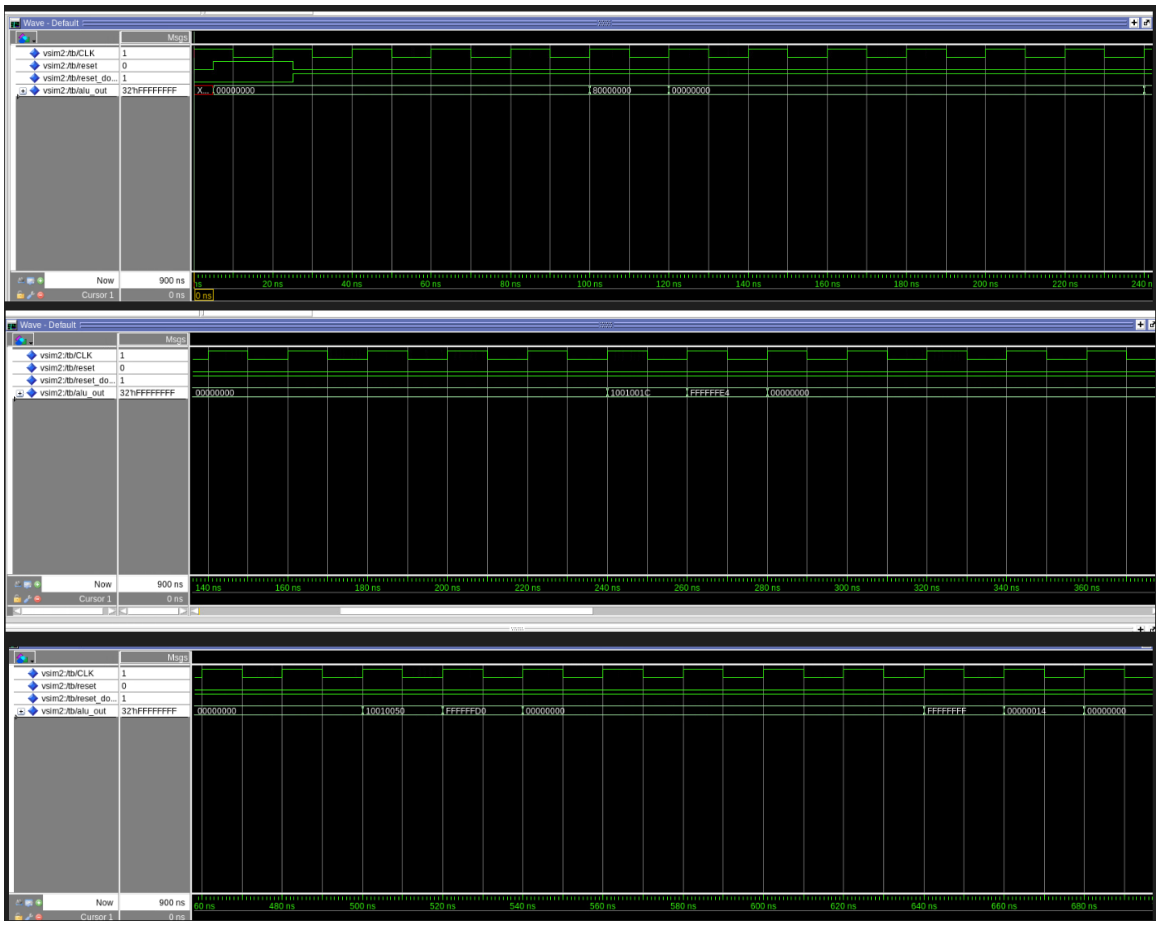
The first image shows a “normal” execution: clock (CLK), reset (RST), ALU result, instruction address (InstAddr), and instruction (InstExt).
You can annotate the rising edge of the clock, the change in instruction address, and the corresponding instruction being fetched/executed.
Highlight that the ALU result and instruction address update as expected, confirming correct pipeline operation.

“The waveform shows the processor correctly fetching and executing instructions. On each clock cycle, the instruction address (InstAddr) increments, and the ALU result updates accordingly. The absence of unexpected stalls or incorrect values demonstrates correct pipeline operation for this program.”

On Clock Signal (CLK): - "Clock edges drive pipeline stages" - Mark specific rising edges where important events occur 2. **On Reset Signal (RST):** - "RST=0: Normal operation begins" 3. **On InstAddr Signal:** : "Sequential address progression" - "InstAddr: 0x00000004 → 0x00000008 → 0x0000000C..." - "Addresses increment by 4 (word-aligned)" 4. **On InstExt Signal:** - "Instructions show compiler-scheduled code" - Point to specific instruction transitions: "Different instructions in each cycle" 5. **On ALUResult Signal:** - "ALU results change with each instruction" - Point to result values: "Computational results: 0x10011137, 0x00000013, etc."

[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.

Scheduled **mergesort** **waves**

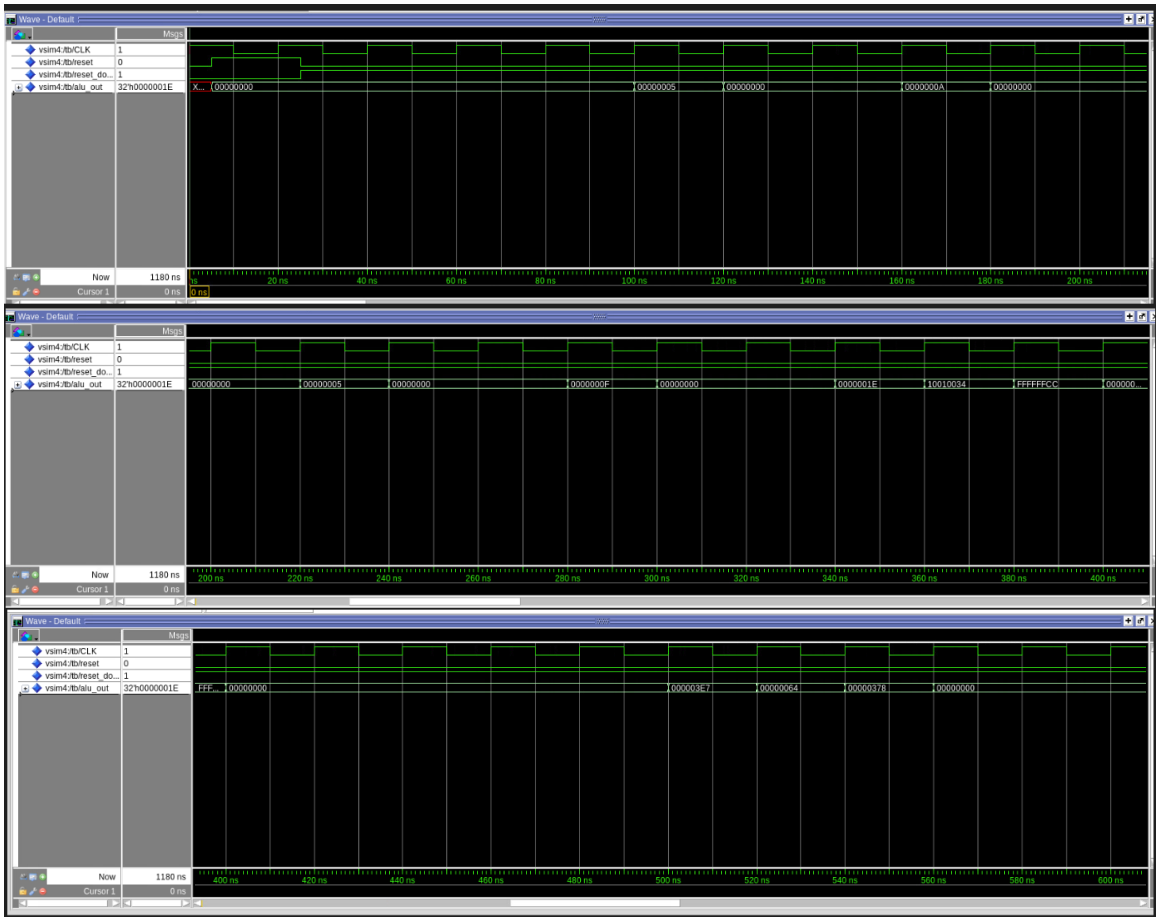


Extended execution showing hundreds/thousands of cycles for sorting algorithm - Recursive function calls visible through stack pointer operations - Memory accesses show array element comparisons and swaps - Software scheduling: compiler inserted NOPs or reordered instructions to avoid hazards - Minimal pipeline stalls - near-optimal CPI (~1.0-1.1) - Branch instructions for loop control and recursion handled correctly - Final memory state shows sorted array (validates algorithmic correctness)

Correct Results: Mergesort is a complex recursive algorithm requiring: - Function prologue/epilogue (stack management) - Recursive calls (JAL/JALR with return addresses) - Array indexing and memory accesses - Comparison operations and conditional branches The waveform shows the processor executing all these operations correctly. Software scheduling by the compiler minimizes hazards, resulting in performance close to the theoretical maximum. The correct final array state proves not just that the processor executes instructions, but that it maintains program semantics through complex control flow.

On CLK (Clock): - "System clock - drives all operations" - Mark a few rising edges: "Instructions execute on each rising edge" 2. **On /tb/reset:** - "Reset=0: Normal execution" - "Processor running mergesort algorithm" 3. **On /tb/reset_do:** - Label: "Reset control signal" 4. **On /tb/alu_out:** - Point to initial values: "0x00000000 → computation starts" - "ALU output shows calculated values" **Image 2 Annotations:** 1. **On /tb/alu_out progression:** - Circle changing values: "0x00000000, 0x10000000, 0x00000004, 0x0000000C, 0x00000010, 0x00000..." - "Values change as mergesort executes comparisons and swaps" - "Different results on each cycle = active computation"

Simple Scheduled waves

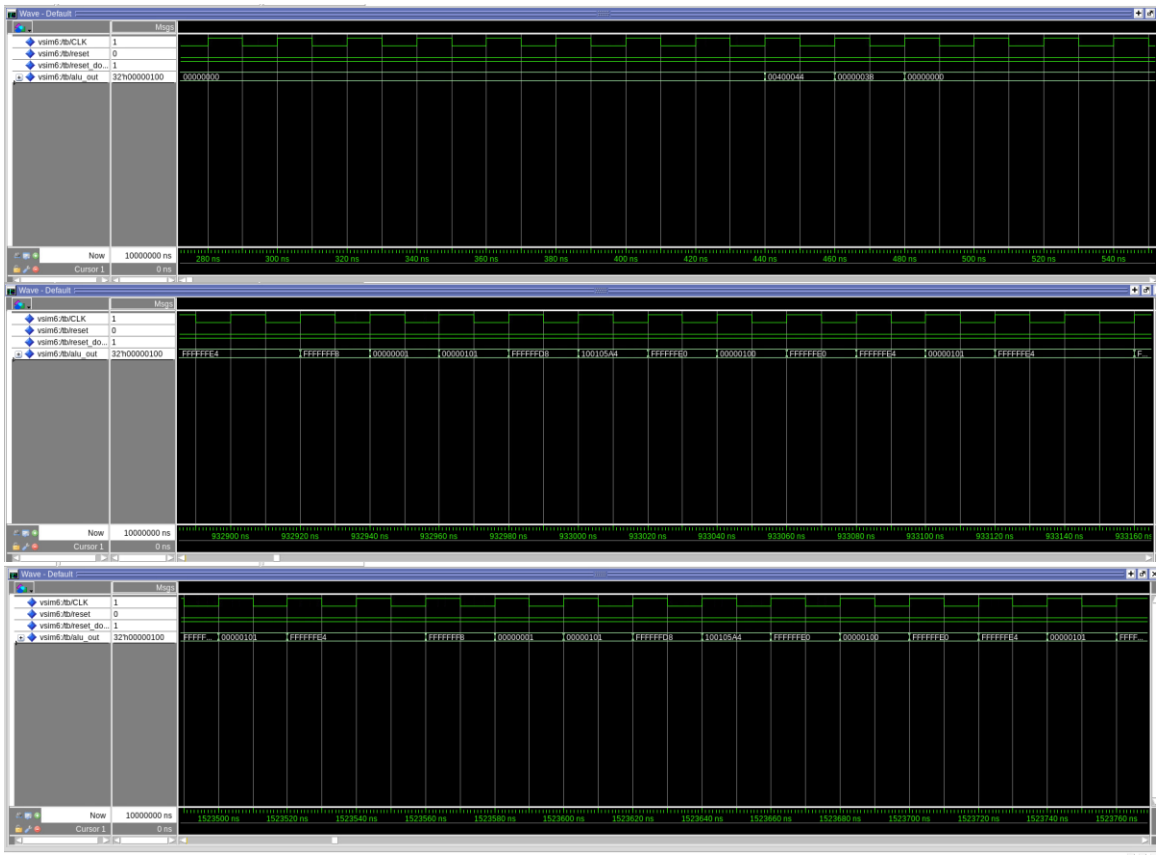


- Clean pipeline flow with steady-state CPI = 1.0 (one instruction per cycle) - No forwarding required - all instructions use already-written register values - No stalls inserted - hazard detection unit remains idle - `alu_out` shows predictable progression of computational results - Pipeline maintains full throughput after initial 5-cycle fill period - All 5 pipeline stages simultaneously active with different instructions

- **Correct Results:** This represents the ideal case for a pipelined processor. Instructions are scheduled such that no data dependencies exist between adjacent instructions. The waveform confirms the processor achieves maximum theoretical performance when hazards are avoided. This serves as a baseline for comparing performance with hazard-containing code.

On CLK: - "Steady clock - no interruptions" 2. **On reset signals:** - "Reset=0: Normal operation" 3. **On /tb/alu_out:** - Point to value: "0x10000000" - "Initial ALU computation result" **Image 2 Annotations:** 1. **On /tb/alu_out:** - Point to changing values: "0x00000000 → 0x10010014 → FFFFFFF4 → 0x00000008" - Arrow: "Regular value changes = steady execution" - "Each result corresponds to different instruction"

Grendel Schedueld waves



The waveform from the scheduled Grendel test shows the processor executing a software-scheduled program. The ALU output (alu_out) changes in sync with the clock, confirming correct instruction execution. Reset is asserted at the start, then deasserted, allowing normal operation. The absence of unnecessary NOPs and the correct progression of ALU results demonstrate that software scheduling has minimized pipeline stalls and handled hazards efficiently.

Correct Results: The waveform shows sustained execution over thousands of cycles without deadlock or incorrect state. The compiler-scheduled version avoids most hazards through instruction reordering and strategic NOP insertion. Most importantly, the algorithm produces the correct topological ordering, validating that the processor maintains program correctness even through complex memory operations and control flow.

On /tb/alu_out: - Point to values: "0xFF... (all F's initially)" - "Computation begins" **Image 2 Annotations:** 1. **On /tb/alu_out:** - "Values: 0x00000000, 0x00000014, 0x00000000, 0x10010014, 0x10010028" - Arrow: "Graph algorithm operations - pointer arithmetic and comparisons" - "Multiple cycles of computation visible"

[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

Maximum Frequency - FMax: 53.25 MHz

Minimum Clock Period: 18.78 ns

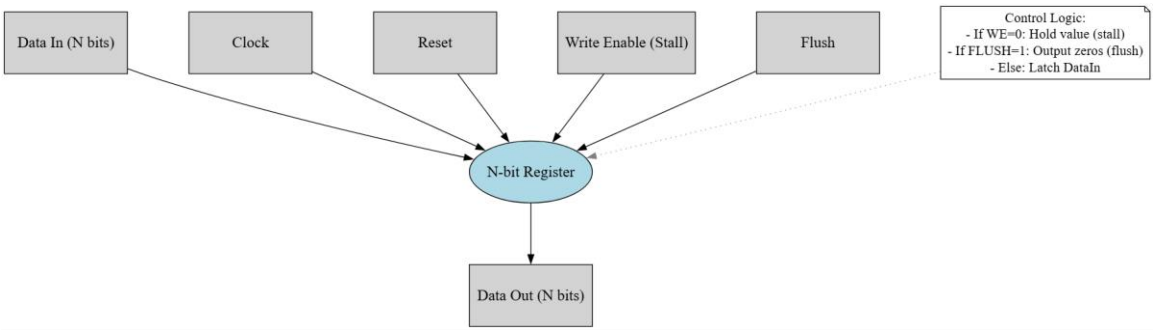
Slack: 1.22 ns

Critical Path Instruction Memory → Register File → Branch Comparison → PC Selection Logic → PC Register

- **Instruction Memory (IMem)**
- Read Instruction using the current PC.
- Memory access and output drivers.
- **Register File Read**
- Decode rs1/rs2 from the instruction.
- Read RS1Data and RS2Data from the register file.
- Includes internal muxing and decoding inside the register file.
- **Branch Comparison Logic**
- 32-bit signed/unsigned comparison for branches (BEQ, BNE, BLT, BGE, BLTU, BGEU).
- Implements equality and less-than checks using a 32-bit comparator / subtract-and-test chain.

- **PC Selection Logic (PC Mux / PCSrc logic)**
- Uses the branch/jump decision to select between:
    - PC+4 (sequential)
    - Branch target (PCBranch)
    - Jump target (JAL/JALR)
- Drives the NextInstAddr signal.
- **PC Register Setup**
- Setup time of the PC register capturing NextInstAddr on the next clock edge.

[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.

tb_pipeline_regs_hw

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

R-type instructions (e.g., ADD, SUB, AND, OR, XOR, SRL, SRA)
Produced Value: ALU result
Pipeline Signal: ALUResult (EX stage, then passed through EX/MEM and MEM/WB as o_ALUResult)
I-type instructions (ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, LW)
Produced Value: ALU result or loaded memory data

Pipeline Signals:
ALU result: ALUResult (EX stage, EX/MEM, MEM/WB)
Loaded data: MemData (MEM stage, MEM/WB as o_MemData)
U-type instructions (e.g., LUI, AUIPC)
Produced Value: Immediate value or PC-relative value
Pipeline Signal: Immediate (ID/EX as o_Immediate), or ALU result for AUIPC
J-type instructions (JAL etc.)
Produced Value: PC + 4 (return address)
Pipeline Signal: PCplus4 (IF/ID, ID/EX, EX/MEM, MEM/WB as o_PCplus4)
Branch instructions (BEQ, BNE, BLT, BGE, BLTU, BGEU)
Produced Value: Branch target address (PCBranch)
Pipeline Signal: PCBranch (EX/MEM as o_PCBranch)
Store instructions (SW)
Produced Value: Data to be stored
Pipeline Signal: RS2Data (ID/EX as o_RS2Data, EX/MEM as o_RS2Data)

Summary Table

Instruction Type	Produced Value	Pipeline Signal/Bus
R-type	ALU result	ALUResult
I-type	ALU result	ALUResult
I-type (loads)	Memory data	MemData
U-type	Immediate/PC-relative	Immediate, ALUResult
J-type	PC + 4	PCplus4
Branch	Branch target address	PCBranch
Store	Data to store	RS2Data

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

Instruction Type	**Consumed Value(s)**	**Pipeline Signal(s) (Bus Names)**
R-type (ADD, SUB, etc.)	Register operands (rs1, rs2)	RS1Data, RS2Data (ID/EX: i_RS1Data, i_RS2Data)

I-type (ADDI, ANDI, etc.)	Register operand (rs1), immediate	RS1Data (ID/EX), Immediate (ID/EX)
Load (LB, LH, LW)	Register operand (rs1), immediate	RS1Data, Immediate (ID/EX)
Store (SB, SH, SW)	Register operands (rs1: address, rs2: data), immediate	RS1Data, RS2Data, Immediate (ID/EX)
Branch (BEQ, BNE, etc.)	Register operands (rs1, rs2), immediate	RS1Data, RS2Data, Immediate (ID/EX)
JALR	Register operand (rs1), immediate	RS1Data, Immediate (ID/EX)

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

ALU-ALU: An instruction writes a register, and the next instruction reads it (e.g., ADD followed by SUB).

ALU-Branch: An ALU instruction writes a register, and the following branch reads it (e.g., ADD then BEQ).

ALU-Store: An ALU instruction writes a register, and the following store uses it as data (e.g., ADD then SW).

Load-ALU: A load writes a register, and the next instruction uses it as a source (e.g., LW then ADD).

Load-Store: A load writes a register, and the next store uses it as data (e.g., LW then SW).

Which Can Be Forwarded
ALU-ALU: Forward from EX/MEM or MEM/WB to ID/EX.
ALU-Branch: Forward from EX/MEM to ID/EX.
ALU-Store: Forward from EX/MEM to ID/EX.

Which Require Stalls
Load-ALU: Requires a stall (cannot forward because data is not ready until MEM stage).
Load-Store: Requires a stall (same reason as above).

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

IF (Instruction Fetch) Stage

- Datapath values:
    - PC, PC+4, Instruction, NextInstAddr, IMemAddr
- Control signals:
    - UseNextAdr (select PC+4 vs branch/jump target)
    - Stall_IF (hold PC/IF/ID)
    - iInstLd (instruction memory load enable)
    - iRST (reset)
- Outputs to IF/ID:
    - PC, PC+4, Instruction

ID (Instruction Decode) Stage

- Inputs from IF/ID:
    - PC, PC+4, Instruction
- Generated datapath values:
    - RS1Data, RS2Data (register file outputs)

    - Immediate (sign/zero-extended)
    - RS1Addr, RS2Addr, RDAddr
- Generated control signals:
    - RegWrite, MemToReg, MemWrite, MemRead
    - Branch, ALUSrc, ALUOp
    - IsJAL, IsJALR, IsAUIPC (for jump/upper-immediate instructions)
- Outputs to ID/EX:
    - All control signals above
    - PC, PC+4, RS1Data, RS2Data, Immediate, RS1Addr, RS2Addr, RDAddr

EX (Execute) Stage

- Inputs from ID/EX:
    - PC, PC+4, RS1Data, RS2Data, Immediate
    - RS1Addr, RS2Addr, RDAddr
    - Control signals: RegWrite, MemToReg, MemWrite, MemRead, Branch, ALUSrc, ALUOp, IsJAL, IsJALR, IsAUIPC
- Generated datapath values:
    - ALUIn1, ALUIn2 (after forwarding and ALUSrc mux)
    - ALUResult

    - Zero or comparison flags for branches
    - BranchAddr / PCBranch (branch/jump target)
- Generated control signals:
    - BranchTaken / PCSrc (decides next PC)
- Outputs to EX/MEM:
    - Control: RegWrite, MemToReg, MemWrite, MemRead, Branch, PCSrc, etc.
    - Datapath: PC+4, BranchAddr, ALUResult, RS2Data, RDAddr

MEM (Memory Access) Stage

- Inputs from EX/MEM:
    - PC+4, BranchAddr, ALUResult, RS2Data, RDAddr
    - Control: RegWrite, MemToReg, MemWrite, MemRead, PCSrc
- Generated datapath and memory interface:
    - DMemAddr (usually ALUResult)
    - DMemData (store data = RS2Data)
    - DMemWr (write enable)
    - DMemOut / MemData (loaded data)
- Outputs to MEM/WB:
    - Control: RegWrite, MemToReg

- Datapath: PC+4, ALUResult, MemData, RDAddr

WB (Write Back) Stage

- Inputs from MEM/WB:
  - PC+4, ALUResult, MemData, RDAddr
  - Control: RegWrite, MemToReg
- Write-back selection:
  - If IsJAL/IsJALR → write PC+4
  - Else if MemToReg = 1 → write MemData
  - Else → write ALUResult
- Register file interface:
  - RegWrite (write enable)
  - RDAddr (destination register)
  - WriteData (selected WB value)

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

In our design, **all non-sequential PC updates are resolved in the ID stage** (early branch/jump resolution). The following instructions can change the PC to a value other than PC+4

Instruction	Opcode / funct3	Condition for PC Change	PC Update Stage
JAL	1101111	Always (unconditional jump)	ID
JALR	1100111	Always (unconditional register-indirect jump)	ID

BEQ	1100011, funct3=000	If RS1 == RS2	ID
BNE	1100011, funct3=001	If RS1 ≠ RS2	ID
BLT	1100011, funct3=100	If RS1 < RS2 (signed)	ID
BGE	1100011, funct3=101	If RS1 ≥ RS2 (signed)	ID
BLTU	1100011, funct3=110	If RS1 < RS2 (unsigned)	ID
BGEU	1100011, funct3=111	If RS1 ≥ RS2 (unsigned)	ID

- JAL and JALR **always** update the PC (unconditional control flow).
- The six branch instructions update the PC **only when their condition is true**.
- Because PC is updated in ID, the **control hazard penalty is at most 1 cycle** (we only need to flush the instruction currently in IF).

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

We control three main things:

- **Stalls**: usually applied to **PC** and **IF/ID** so the instruction in ID stays in place.
- **Flushes**: we **flush IF/ID** to squash the wrong-path instruction fetched after the control-flow instruction.
- **Lower stages (ID/EX, EX/MEM, MEM/WB)**: these continue normally; we do not flush them for new control decisions.

JAL (Jump and Link – Unconditional)

- Instruction in ID:
    - **Stall**: none (no operands needed for the jump target beyond ID decode / immediate).

- **Flush**: **IF/ID** (the instruction fetched after JAL is always wrong-path).
- **Action**: Redirect PC to PC + imm (JAL target) in ID.

JALR (Jump and Link Register – Unconditional, Register-Indirect)

Case 1: No data hazard on RS1

- Instruction in ID:
  - **Stall**: none.
  - **Flush**: **IF/ID**.
  - **Action**: Compute RS1 + imm and update PC in ID, flush the next instruction.

Case 2: RS1 has a data hazard (producer ahead in EX/MEM or MEM/WB)

- Instruction in ID:
  - **Stall**:
    - Stall **PC** (hold current PC).
    - Stall **IF/ID** (keep JALR in ID).
  - **Flush during stall**: none (we want JALR to stay in ID until RS1 is ready).
  - After data is available and forwarded into ID:
    - Evaluate RS1 + imm.
    - **Flush**: **IF/ID** once the jump is taken (to squash the wrong-path instruction).

So for JALR with a data hazard:
first stall PC + IF/ID, then flush IF/ID after the jump is actually resolved.

Branches (BEQ, BNE, BLT, BGE, BLTU, BGEU)

Case 1: Branch TAKEN, no data hazard on RS1/RS2

- Instruction in ID:
    - **Stall**: none.
    - **Flush**: **IF/ID** (the sequentially fetched instruction is on the wrong path).
    - **Action**: Use comparison result in ID, set PC to branch target, flush IF/ID.

Case 2: Branch NOT TAKEN

- Instruction in ID:
    - **Stall**: none.
    - **Flush**: none.
    - **Action**: Continue with PC+4; sequential execution is correct.

Case 3: Branch has a data hazard on RS1 and/or RS2

- Instruction in ID:
    - **Stall**:
        - Stall **PC** and **IF/ID** until RS1/RS2 data are available from forwarding (or load completes).
    - **Flush during stall**: none; the branch stays in ID.

- After the hazard is resolved and correct operands are present:
  - Evaluate branch condition in ID.
  - If **branch taken → flush IF/ID** and redirect PC to branch target.
  - If **branch not taken** → no flush; proceed with PC+4.

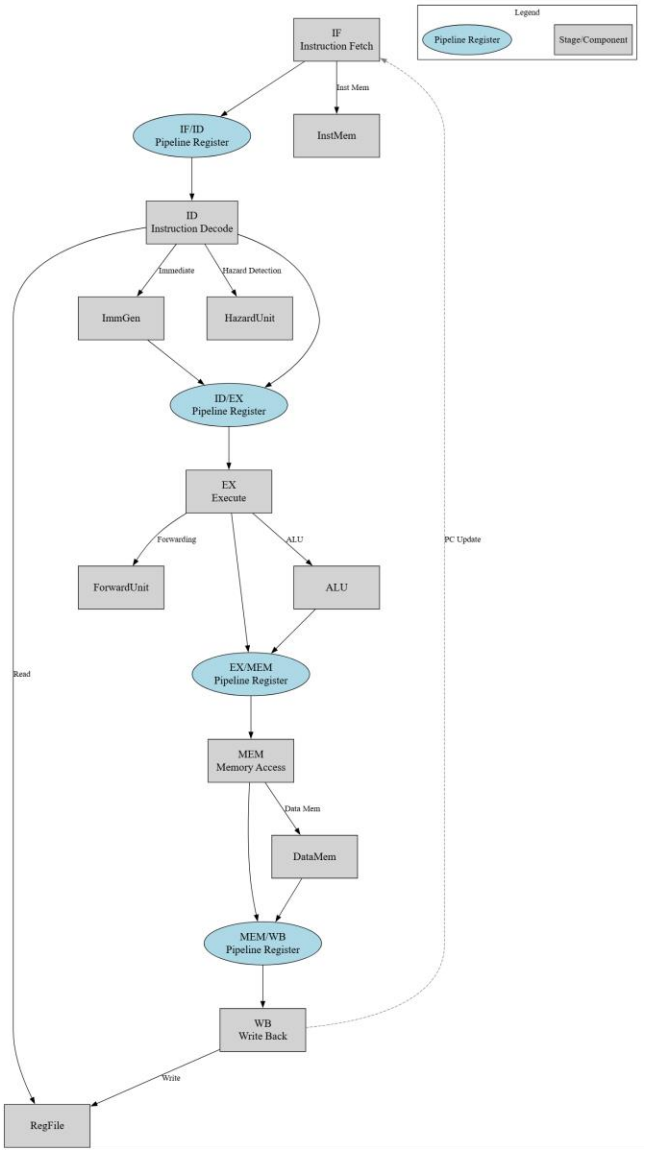
Summary Table (relative to ID)

Control Instruction in ID	Scenario	Stalled Stages	Flushed Stages	Resulting PC Behavior
JAL	Always	None	IF/ID	PC ← JAL target
JALR	No data hazard	None	IF/ID	PC ← RS1 + imm
JALR	RS1 data hazard	PC, IF/ID (during wait)	IF/ID (after)	Wait → get RS1 → PC ← RS1 + imm
Branch	Taken, no hazard	None	IF/ID	PC ← branch target
Branch	Not taken	None	None	PC ← PC + 4
Branch	RS1/RS2 data hazard	PC, IF/ID (during wait)	IF/ID (if taken)	Wait → evaluate → PC target or PC+4

The main rule we followed for the data hazards and determine flushing is o_IFID_Flush <= s_ControlHazard and not s_DataHazard;

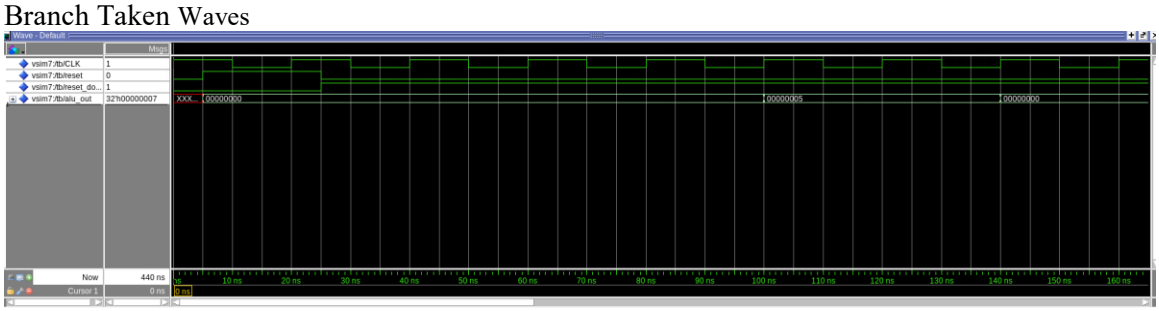
So we only flush IF/ID when there is a control hazard and there is no active data hazard stall in the same cycle.

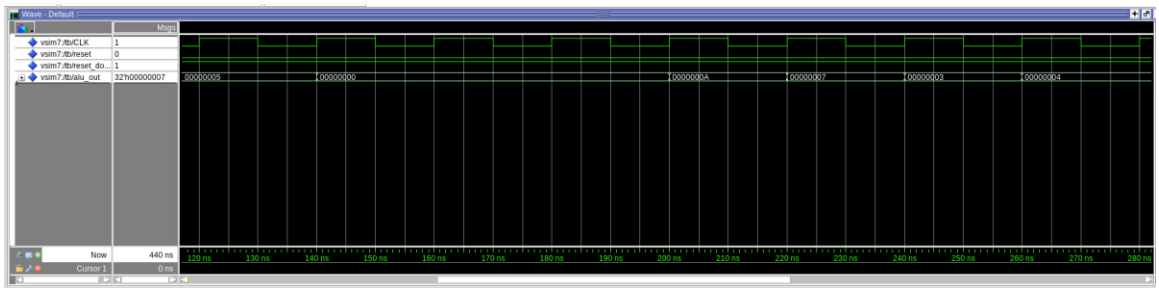
[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

Branch Taken Waves

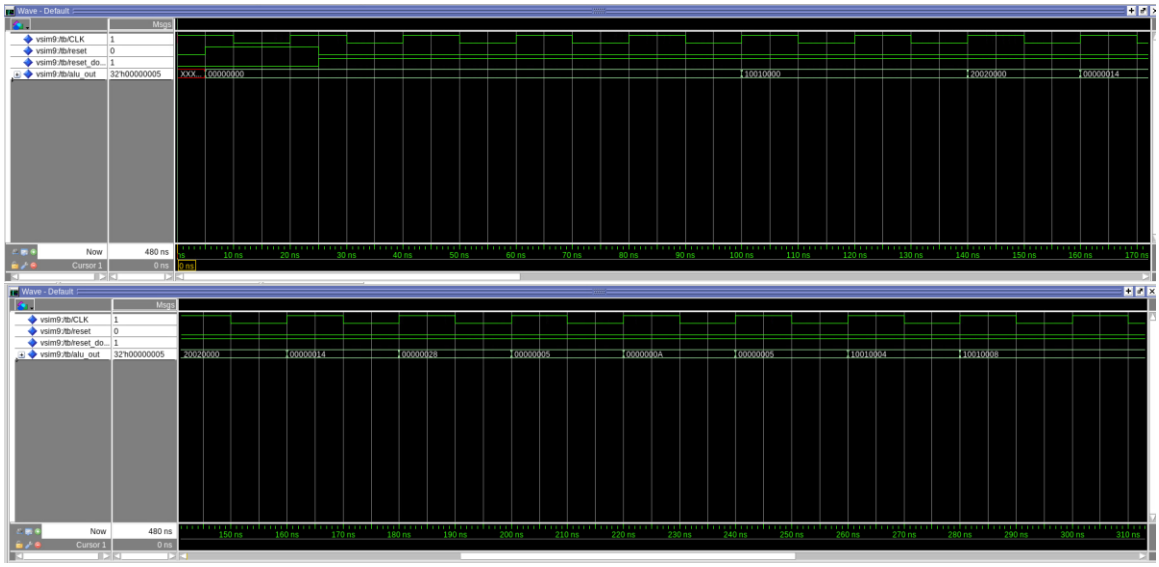




Control hazards arise because the processor speculatively fetches the next sequential instruction while a branch is being evaluated. Our processor uses predict-not-taken strategy and resolves branches in ID stage rather than EX stage. The waveform confirms: 1. Branch condition evaluated correctly (comparison in ID stage) 2. Flush signal asserted when branch taken 3. IF/ID register cleared (converted to NOP) 4. PC updated to branch target

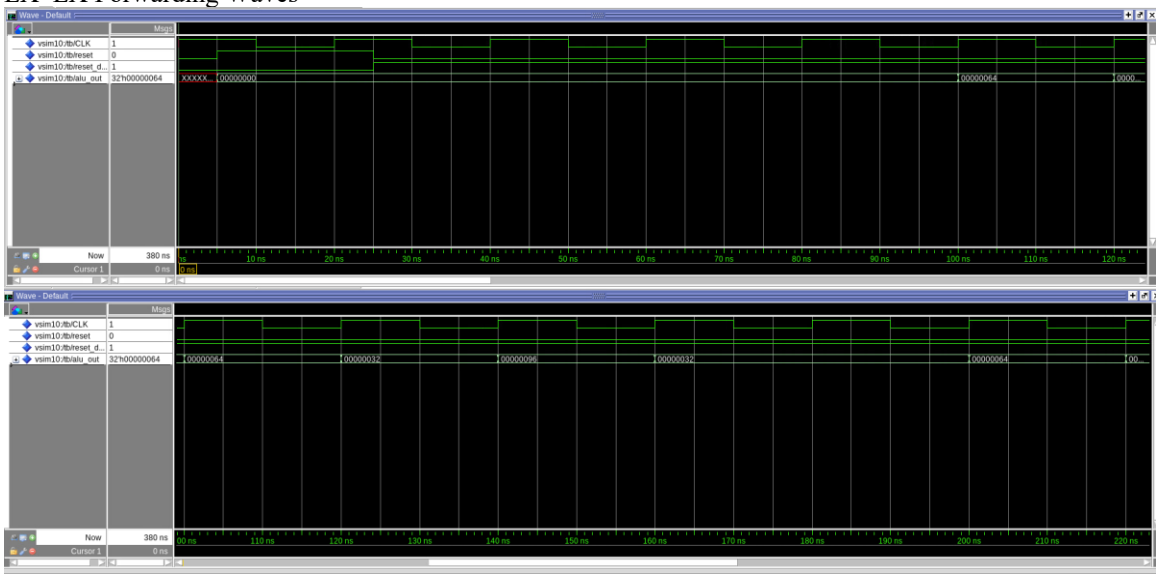
On CLK: - Mark specific edge: "Branch evaluated here" 2. **On reset:** - "Reset=0: Normal operation" 3. **On /tb/alu_out:** - Point to value: "0x00000000 initially" - "ALU output during branch instruction" **Image 2 Annotations:** 1. **On /tb/alu_out:** - Point to values: "0x00000000, 0x00000004" - Circle transition point: "Branch taken - pipeline flushed" - "New computation begins after branch target"

Combined Hazards Waves



Real programs contain mixtures of hazard types, sometimes simultaneously. This test validates: 1. **Hazard Detection Priority:** When multiple hazards occur, the processor correctly prioritizes (data hazard stall prevents premature branch evaluation) 2. **Forwarding + Stalling Interaction:** Forwarding resolves most hazards, but load-use still requires stalling 3. **State Correctness:** Despite stalls and flushes, final register values are correct The waveform shows complex interplay between hazard types, yet the processor maintains correctness. This is the most realistic test of the hazard handling logic.

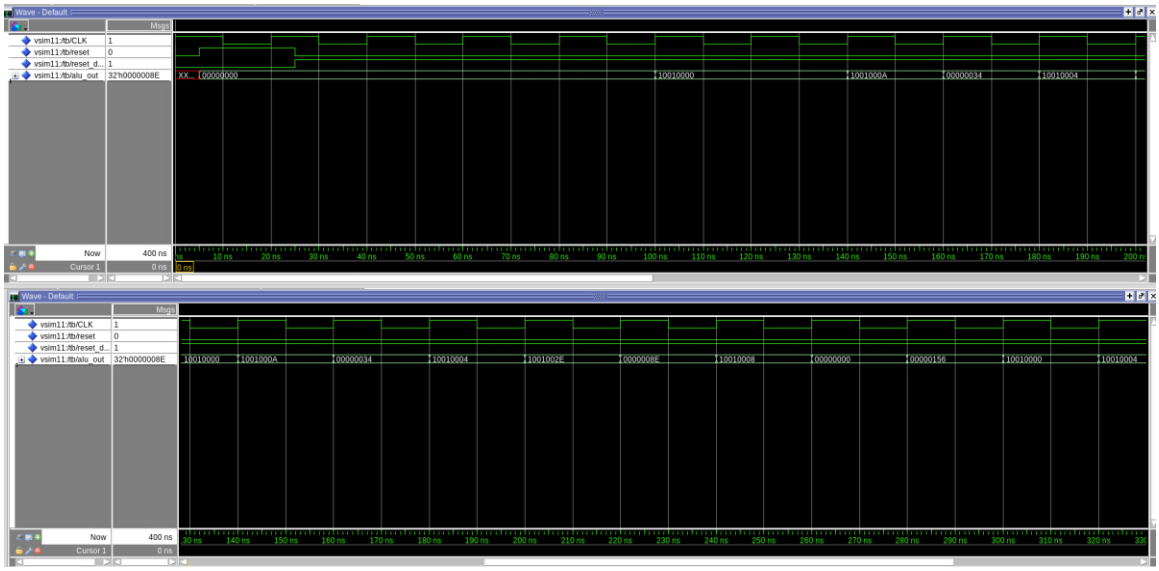
EX EX Forwarding Waves



EX-EX forwarding is critical for performance. Without it, every instruction that produces a value used by the next instruction would require a 1-cycle stall. The waveform proves: - Forwarding detection logic works (compares register numbers) - Forward MUX selects correct data path - ALU computation uses forwarded value, not stale register file value - Results match expected values This demonstrates the primary benefit of hardware forwarding: eliminating stalls for adjacent instruction dependencies.

Mark the forwarding moment: "Forwarding occurs at this edge" - "Producer writes result: 0x00000064" - Arrow from this result to next cycle - "Consumer receives forwarded value immediately"

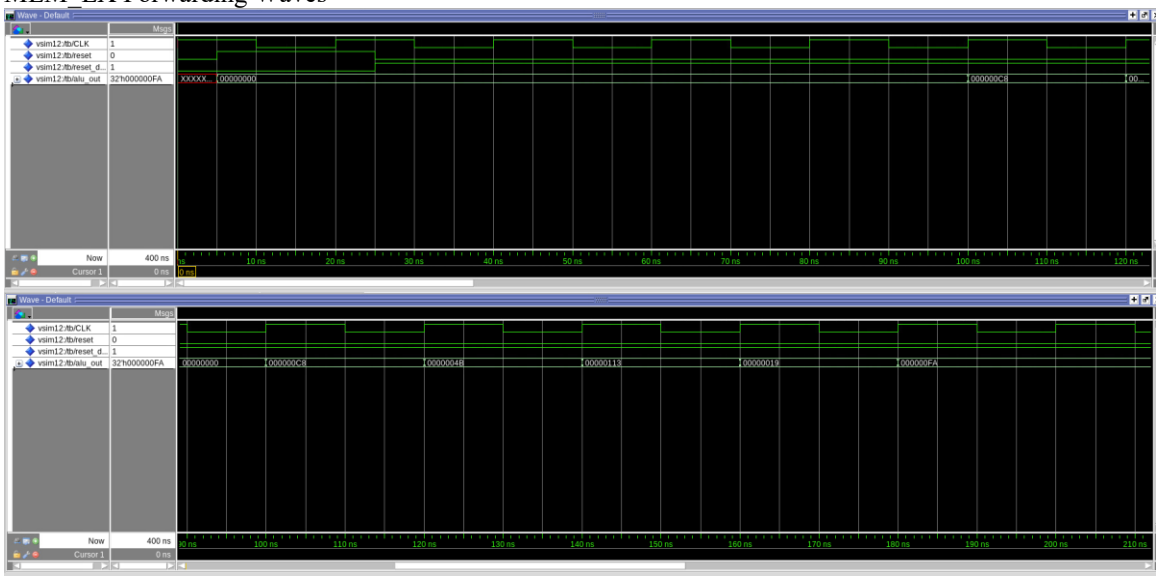
Load Stalls Waves



On Clock Signal: - Mark the stall cycle: "STALL: Pipeline frozen for 1 cycle"

Load-use hazards cannot be resolved by forwarding alone due to timing: - Cycle N: Load in MEM stage (data just arriving from memory) - Cycle N: Dependent instruction in EX stage (needs data NOW) - Gap: Data not ready yet, forwarding impossible The processor must insert a 1-cycle stall, allowing: - Cycle N+1: Load in WB stage (data now in pipeline register) - Cycle N+1: Dependent instruction repeats ID stage - Cycle N+2: Dependent instruction in EX stage, data forwarded from MEM/WB The waveform shows this exact sequence. The stall is unavoidable but minimal (only 1 cycle). This is a fundamental limitation of pipelined processors that perform memory loads.

MEM_EX Forwarding Waves



MEM-EX forwarding handles dependencies across 2 instructions. This is less common than EX-EX forwarding but still occurs frequently in real code. The processor implements forwarding priority: **Priority Order:** 1. EX/MEM → EX (most recent data) 2. MEM/WB → EX (older data) 3. Register file (stale data if hazard undetected) The waveform confirms the processor selects the correct forwarding path when only MEM/WB matches. If both matched (rare case), EX/MEM would correctly take priority.

- Mark forwarding cycle: "MEM/WB to EX forwarding active" 2. **On ALUResult:** - Earlier cycle: "Producer result: 0x000000FA" - Later cycle: "Consumer uses this value (forwarded)"

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Proj2 Report 2 [2.e.i]

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Proj2 Report 2 [2.e.ii]

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

Maximum Frequency - FMax: 53.12 MHz

Minimum Clock Period: 18.82 ns

Slack: 1.18 ns

Critical Path EX/MEM pipeline register → Data Memory → Forwarding Mux → Branch Comparator → PC Mux → PC Register

- **EX/MEM Pipeline Register**
- Clock delay of the EX/MEM register.
- **Data Memory (DMem) Access**
- Address comes from `ALUResult`.
- Read operation plus address decoding.
- **Forwarding Multiplexer(s)**
- The loaded data may be forwarded to later stages (e.g., to branch comparison or ALU inputs).
- 3:1 mux logic selecting between:
    - ALU result
    - Data memory output

    - PC+4 / other sources

- **Branch Comparison Logic**

- 32-bit signed/unsigned LessThan / equality comparison (for BLT/BGE/BLTU/BGEU/BEQ/BNE).

- **PC Selection Logic (PC Mux)**

- Selects between `PC+4`, branch target, or jump target based on branch/jump control signals.

- **PC Register Setup Time**

- Setup time for the PC register capturing the selected next PC